
# DealHunter

# Deliverable 2

---

**Computer Engineering - 3rd year**

**Members of the project:** Otto Biel Ródenas, Sergi Arbós Nicolau, Marc Verdugo Soria, Bruno Verdugo Soria, Marc Prats, Joel Marcet Cruz

**Date:** 14 / 05 / 2026

**Subject:** Web Project

**Teachers:** Roberto Garcia González and David Sarrat González

**Repository:** https://github.com/DealHunterOrgan/DealHunter.git

# Índex

# 1. Design and Decision Considerations

## 1.1 Creation, edition and deletion of a model instance

In order to allow users to create, edit and delete some instances of our model, we have chosen a simple yet very useful and informative option: Reviews.

So, to enable this, we created a new model in our models.py file, which initially didn't have a review field, because we didn't think of this function at the beginning. It has the following structure:

```
class Review(models.Model):  12+ usages  sergiarbos
    game = models.ForeignKey(Game, on_delete=models.CASCADE, related_name='reviews')
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    content = models.TextField(max_length=1000)
    rating = models.IntegerField(default=5)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:  sergiarbos
        ordering = ['-created_at']

    def __str__(self):  sergiarbos
        return f"Review by {self.user.username} on {self.game.title}"
```

### Design

- **ForeignKey(Game, on_delete=CASCADE)** -> Each review is tightly coupled to a game. If the game is removed from the database then all associated reviews are automatically deleted, avoiding orphaned records.
- **ForeignKey(User, on_delete=CASCADE)** -> If a user deletes their account their reviews are removed too. This also lets us enforce ownership checks (Review.objects.filter(user=request.user)).
- **content = TextField(max_length=1000)** -> A free-text comment field capped at 1000 characters to prevent abuse while still allowing meaningful feedback.
- **rating = IntegerField(default=5)** -> A numeric 1–5 star rating matching the selector presented in the UI. The default of 5 ensures that a submitted form without an explicit rating is never invalid.

- **auto_now_add=True on created_at** -> The timestamp is set automatically on creation and cannot be tampered with by the user.
- **ordering = ['-created_at']** -> The default queryset ordering shows the most recent reviews first, which is the expected UX pattern for comment sections.

## 1.2 How does this functionality work?

They basically (when logged in) are able to valorate, with a 5-star system and leaving a comment if they want, making a stronger community and providing other users a more accurate idea about the game they want to buy. Of course, because everyone can change their mind or make mistakes, this feature is enabled with the possibility of edition or deletion by the user who made that review. Below, is briefly explained how each of the functionalities work in our web.

### Creation:

On each game's detail page a logged-in user sees a comment form rendered directly below the media section. The form contains a select dropdown with star ratings from 1 to 5, a textarea for the written comment and a "Post comment" submit button.The form POSTs, which is handled by AddReviewView. A LoginRequiredMixin-protected CreateView. The view automatically assigns game and user from the URL parameter and the active session respectively. On success, the user is redirected back to the game's detail page where the new review appears at the top of the list.

### Edition:

Each review displayed on the detail page shows an edit button only for the review's author. Clicking it toggles an inline edit form that pre-populates the current rating and content. The user can modify the text and the star rating. Then click "Save".

An UpdateView that restricts its queryset to Review.objects.filter(user=request.user). This means that even if a user manually crafts a POST request to another user's review URL, the server returns a 404 because the object is not found in the filtered queryset.

After saving, the view checks the HTTP Referer header: if the edit was initiated from the profile page, the user is redirected back there. Otherwise they return to the game's detail page.

### **Deletion**:

Each review authored by the logged-in user displays a delete button alongside the edit button. Pressing it triggers a JavaScript confirmation dialog before submitting a POST request.

Like the edit view, DeleteReviewView filters its queryset to user=request.user, so a user cannot delete someone else's review even with a direct URL. The same Referer-based redirect logic applies.

On the profile page, the three most recent reviews are also listed in the sidebar with their own individual delete buttons allowing users to manage their reviews from a central location without having to navigate to each game page.

## 2. E2E Testing

For this deliverable, E2E tests have been implemented following the recommended approach using Behave and Splinter with the behave-django integration.

The test environment is configured in environment.py. It creates a django Splinter browser instance, which uses Django's test client under the hood. No real browser is launched, making the tests fast. All required packages are declared in requirements.txt.

The file reviews.feature describes four scenarios covering creation, edition, deletion, and a security restriction:

- User/game creation uses get_or_create so the Background steps are idempotent across test runs.
- Login is performed by visiting the login URL and filling the form fields programmatically, simulating real browser interaction.
- Edit is performed through the inline form identified by its CSS ID, which matches the ID attribute rendered in the template.
- Delete triggers the form with the button[title="Eliminar comentario"].

- Security restriction check asserts that the edit and delete buttons are not present in the DOM when the logged-in user is not the review's author. This validates both the template-level conditional rendering and the server-side queryset restriction.

Here is the step definition that enforces this check:

```
@then('I should not see the Edit or Delete buttons')
def step_should_not_see_buttons(context):
    assert context.browser.is_element_not_present_by_css('button[title="Eliminar comentario"]')
    assert context.browser.is_element_not_present_by_css('button[title="Editar comentario"]')
```

These CSS selectors match the title attributes set in detail.html only inside the {% if r.user == user %} block, so they are guaranteed to be absent for any other logged-in user.

The tests cover:

- Correct behavior: creation, edition, and deletion flows work end-to-end.
- Error / security: a different user cannot see or interact with another user's review controls.

## 3. External or second API incorporation

In addition to the CheapShark API (used in the first deliverable to retrieve game deals and pricing data), we integrated a second external API called RAWG.

RAWG is one of the largest video game databases offering lots of metadata such as descriptions, screenshots, and trailers. CheapShark provides excellent commercial data (prices, deals, store links) but does not supply media assets or long-form descriptions. RAWG fills this gap.

The integration lives in services.py:

- **get_rawg_id_by_title(title)** -> Takes a game title, strips common suffixes and queries the RAWG/games endpoint with search=<title>&page_size=1. Returns the RAWG internal game ID of the top result.

- **get_game_media(game_title)** -> Called from GameDetailView.get_context_data() every time a user opens a game detail page.
    - Fetches the game description from /games/<rawg_id> (using description_raw if available, falling back to description).
    - Fetches up to 4 screenshots from /games/<rawg_id>/screenshots.
    - Fetches a trailer video URL from /games/<rawg_id>/movies.
    - Returns a media dictionary that is passed to the detail.html template.

The RAWG API key is stored in the .env file and loaded via os.getenv('RAWG_API_KEY'), keeping credentials out of the codebase. If the key is not set the function returns empty/default values so the page still renders without errors.

Also the RAWG integration is complemented by an AJAX-powered autocomplete feature on the search bar, which provides a responsive real-time suggestion dropdown as the user types.

```
def game_autocomplete(request):
    query = request.GET.get('q', '').strip()
    results = []
    if len(query) > 1:
        games = Game.objects.filter(title__icontains=query)[:6]
        for g in games:
            results.append({
                'id': g.id,
                'title': g.title,
                'cover': g.cover_url
            })
    return JsonResponse({'results': results})
```

This view is mapped to /autocomplete/ and returns a JSON array of up to 6 matching games (title + cover image URL + ID).

The navbar search bar and the home page search bar both listen to the input event on the text field. On each keystroke, they fire a fetch() call to /autocomplete/?q=<query>. The JSON response is rendered as a styled dropdown list of clickable items, each showing the game cover thumbnail and title. Clicking an

item navigates directly to the game's detail page. The dropdown closes automatically when the user clicks outside of it.

This feature provides a live, server-assisted search experience that significantly improves usability compared to a plain form submission.

# 4. Views.py Modifications

Moreover, we have changed and modified the views.py file, so it follows and achieves the ClassViews and ModelForms standards for the creation, deletion and edition feature.

## 4.1 New CBVs added for the Review CRUD

| View | Description |
|---|---|
| AddReviewView | Creates a new Review: assigns game and user from the URL and session automatically. |
| EditReviewView | Edits a Review: queryset filtered to current user. |
| DeleteReviewView | Deletes a Review. |

All three inherit LoginRequiredMixin as their first base class. This ensures that any unauthenticated request is automatically redirected to the login page providing a clean authentication gate without any manual if not request.user.is_authenticated checks.

**Ownership enforcement** is implemented by overriding get_queryset() in both EditReviewView and DeleteReviewView:

```
def get_queryset(self):
    return Review.objects.filter(user=self.request.user)
```

If someone tries to edit or delete a review they do not own, Django's get_object_or_404 machinery returns a 404 response without leaking information about whether the object exists at all.

For the Review views we used the fields = ['content', 'rating'] shorthand on the CBV instead of defining an explicit ModelForm subclass. Django generates the form automatically from the model fields. The game and user fields are excluded from the form and assigned programmatically in form_valid(), which prevents any user from submitting forged values for those fields.

Both EditReviewView and DeleteReviewView implement a context-aware get_success_url():

```
def get_success_url(self):
    referer = self.request.META.get('HTTP_REFERER', '')
    if 'profile' in referer:
        return reverse_lazy('games:profile')
    return reverse_lazy('games:detail', kwargs={'pk': self.object.game.id})
```

This ensures that a user editing a review from the profile page is sent back to the profile, not to the game detail page providing a smoother UX without needing separate URL patterns for each entry point.

## 5. Other implementations or extensions

- Profile page: The user profile was redesigned for this deliverable. It now shows a stats panel (savings, review count, rank), the full wishlist grid with remove buttons and a "Recent Activity" sidebar listing the 3 most recent reviews with inline delete buttons. A gamification Hunter Level (1–5) is also computed from the total number of wishlist items + reviews.
- Avatar system: Users can choose from 6 different avatar images via the EditProfileForm. The selection is stored in the Profile model.
- Image zoom modal: On the game detail page, clicking any screenshot or cover image opens a full-screen lightbox modal.
- Inline edit form: The review edit form is shown/hidden in place via JavaScript without a page navigation, giving the page a more dynamic web-app feel.
- delete_account_confirm.html template: A confirmation page is shown before permanently deleting the user account, preventing accidental deletions.

# 6. Credentials for testing and relevant users facilitated

- Regular User:

  teacherTest // test1234!

- Super User:

  adminTest // admin1234!

- Other Regular User:

  Otto616 // ottootto